Documentação do Projeto: Algoritmo do Banqueiro

Aluno: Luca Moreira Ribeiro Mazala de Araujo

Repositório no GitHub

O código fonte deste projeto está disponível em um repositório público no GitHub: https://github.com/lucam05/Traballho-de-Sistemas-Operacionais

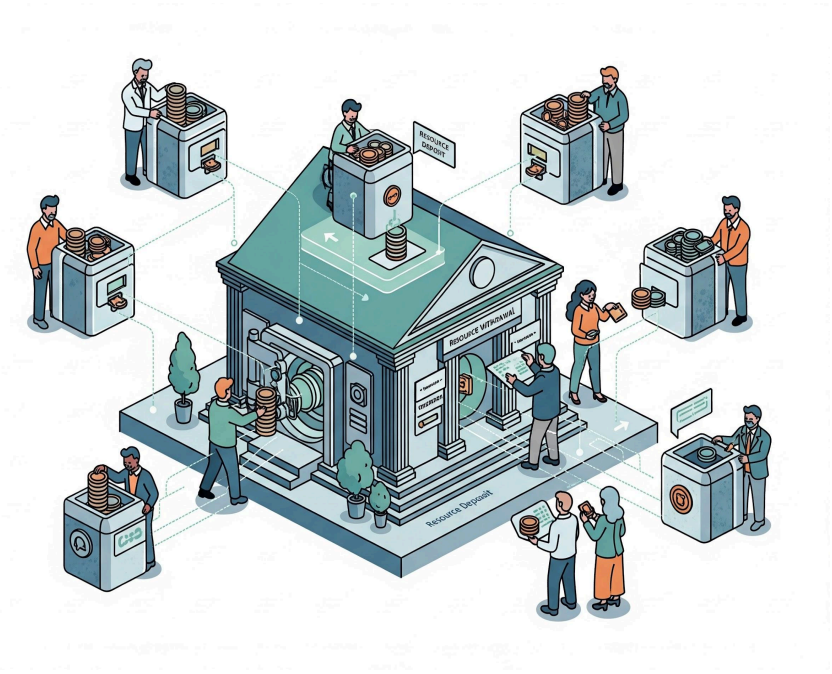
O repositório inclui:

- Código fonte completo.
- Arquivo `README.md` com instruções detalhadas de compilação e execução.
- Estrutura do projeto Maven.

Para clonar o repositório: `git clone https://github.com/seu-usuario/algoritmo-banqueiro.git`

Descrição do Projeto

Este projeto implementa o Algoritmo do Banqueiro em Java, como descrito na Seção 7.5.3 do livro "Fundamentos de Sistemas Operacionais" de Abraham Silberschatz et al. O programa simula um sistema multithreaded onde múltiplos clientes solicitam e liberam recursos controlados por um "banqueiro" que garante que o sistema permaneça em um estado seguro, evitando deadlocks.



Funcionalidades Principais

- **Controle de Recursos**: Gerencia 3 tipos de recursos com quantidades iniciais fornecidas via linha de comando.
- **Clientes Multithreaded**: 5 threads de clientes que continuamente solicitam ou liberam recursos de forma aleatória.
- **Algoritmo de Segurança**: Verifica se uma solicitação deixa o sistema em um estado seguro antes de aprová-la.
- **Sincronização**: Usa blocos `synchronized` para prevenir condições de corrida.
- **Logs de Operações**: Exibe no console as solicitações e liberações com status de sucesso ou falha.

Estrutura do Código

- **Classe Main**: Contém as estruturas de dados globais (`available`, `maximum`, `allocation`, `need`), métodos `request_resources` e `release_resources`, algoritmo de segurança, e a classe interna `Customer`.
- **Classe Customer**: Extende `Thread` e simula o comportamento de um cliente, solicitando ou liberando recursos aleatoriamente.

Estruturas de Dados

Variável	Tamanho	Descrição
`available`	`[3]`	Quantidade disponível de cada recurso.
`maximum`	`[5][3]`	Demanda máxima de cada cliente.
`allocation`	`[5][3]`	Recursos atualmente alocados a cada cliente.
`need`	`[5][3]`	Recursos ainda necessários para cada cliente (`need = maximum - allocation`).

Como Compilar e Executar

Pré-requisitos

- Java JDK 17 ou superior instalado.
- Sistema operacional Windows (ou compatível com comandos `javac`/`java`).

Passos

1. Navegue até o diretório do projeto: `cd "c:\Users\lucam\OneDrive\Área de Trabalho\Traballho-de-Sistemas-Operacionais\trabalho"`
2. Compile o código: `javac -d target/classes src/main/java/so/br/Main.java`
3. Execute o programa com os recursos iniciais: `java -cp target/classes so.br.Main <recurso1> <recurso2> <recurso3>`
    - Exemplo: `java -cp target/classes so.br.Main 10 5 7`

O programa executará indefinidamente, simulando as operações dos clientes. Para interromper, use Ctrl+C.

Algoritmo do Banqueiro

Solicitação de Recursos (`request_resources`)

1. Verifica se a solicitação $\le$ necessidade do cliente.
2. Verifica se a solicitação $\le$ recursos disponíveis.
3. Aloca temporariamente os recursos.
4. Executa o algoritmo de segurança.
5. Se seguro, confirma a alocação; caso contrário, reverte.

Liberação de Recursos (`release_resources`)

1. Verifica se a liberação $\le$ alocação atual do cliente.
2. Libera os recursos de volta para disponíveis.

Algoritmo de Segurança

- Inicializa `work = available` e `finish = false` para todos.
- Enquanto houver clientes não finalizados:
  - Encontra um cliente não finalizado onde `need` $\le$ `work`.
  - Adiciona `allocation` do cliente a `work` e marca como finalizado.
- Se todos forem finalizados, o estado é seguro.

Valores de Configuração

Configuração	Valor
Número de Clientes	5
Número de Recursos	3

As demandas máximas são *hardcoded* (embutidas no código):

Cliente	Recurso 1	Recurso 2	Recurso 3
Cliente 0	7	5	3
Cliente 1	3	2	2

Cliente	Recurso 1	Recurso 2	Recurso 3
Cliente 2	9	0	2
Cliente 3	2	2	2
Cliente 4	4	3	3

Tratamento de Erros

- Solicitações inválidas (excedem necessidade ou disponíveis) são negadas.
- Liberações inválidas (excedem alocação) falham.
- O programa não termina automaticamente; deve ser interrompido manualmente.

Limitações e Melhorias Possíveis

- Demandas máximas são *hardcoded*; poderiam ser lidas de arquivo.
- Não há limite de tempo ou condição de parada.
- Poderia incluir mais logs ou interface gráfica.
- Testes unitários poderiam ser adicionados.

Referências

- SILBERSCHATZ, Abraham; GALVIN, Peter B.; GAGNE, Greg. Fundamentos de sistemas operacionais. 9. ed. Rio de Janeiro, RJ: LTC, c2015.

Documento criado em: Date
Autoeholder-type="person">